The issue is that when one is reviewing thousands of lines of code at a time, a drop in concentration for even a minute can result in a major erroneous statement creeping through. Some of you may counter the above problem by saying, "Not every defect is meant to be detected by means of code reviews and if that is the expectation, then we don't even need unit testing." While it is definitely true that code reviews cannot catch every latent defect in the code, it is reasonable to expect that glaring defects, such as the one above, should be flagged by being very careful during the reviewing process.

Doing code reviews carefully and diligently is a technique each developer needs to cultivate. Requesting the author of the code to provide a brief documentation or write-up summarising what the code change is all about is a useful aid in doing effective code reviews. Not only does this help the reviewer to get a better understanding of the code, it makes developers double-check their own work so that any defects/issues missed out by them earlier can be spotted by themselves. Also note that concentration is important during code reviews. So don't plan to do code reviews over a long stretch, say for more than an hour. Typically, within an hour, one can carefully review around 100-200 lines of production quality code.

Another important aid for effective code reviews is to maintain a checklist of what to look for. Make a note of things you normally check for and use that checklist as a first level filtering tool in your review process. While the checklist is no replacement for the actual visual inspection of the code and understanding the functional part of the change, it can help in catching some of the routine errors/mistakes that can creep in, during development. Here is my code review check-list.

1. Does this change introduce a new feature?
   a. Has the design document been reviewed?
   b. Has the test plan for the feature/enhancement been reviewed?
   c. Are there deviations from design? If so, have they been agreed upon by the architects?
   d. Does this functionality have interaction with other features? If so, have you discussed the possible impact with the teams involved?
   e. If you have, then did you add people from those teams as reviewers?
   f. If you did, have you tested the code in combination with those dependent features?
   g. Have you provided a list of your unit test cases to your reviewers?
   h. Does this change have a performance impact on the overall product? If so, have you quantified the performance impact with the feature on/off and documented it?
   i. Has adequate debug tracing been added?
   j. Have you documented use-case scenarios?
   k. Does your unit testing cover all documented use-cases?
   l. Have you removed trace *printfs* no longer needed in your code?
   m. Do you have any unused functions/variables left over in the code?
2. Is this code change addressing a defect fix?
   a. If so, do you have a reproducer test case for the issue?
   b. Have you verified that the reproducer has been tested and that the problem does not happen again?
   c. Have you verified whether there are other code paths that may have the same issue?
   d. If so, have all those code paths been fixed as well?
   e. Have you provided a list of your unit test cases to your reviewers?
   f. Have you explained the defect triggering scenario to the reviewer, in detail?
   g. Have you explained the fix to the reviewer, in detail?
3. Does this change allocate memory?
   a. If so, what is the size of the allocation and the size of reads/writes to it?
   b. Have you verified that there are no possible buffer overruns (updates don't write past the allocated memory)?
   c. Does the memory get it freed on all error paths as well as normal error paths?
   d. Is this a higher order memory allocation? Can it fail due to fragmentation?
   e. Have you ensured that there are no double frees?
   f. Have you ensured that there are no 'Use After Free' for this allocated memory?
4. Does this change acquire a lock?
   a. If so, check where the lock gets released.
   b. Does it get released on all error return paths as well as on normal return paths?
   c. How big is the critical section in your code?
   d. Did you verify whether or not that critical section is too large (check with reviewer if in doubt)?
   e. Are there potential deadlock situations? Review all other users of this lock and ensure that no lock ordering issues exist.
5. Does this checkin involve any integer arithmetic?
   a. If so, have you checked for potential overflow issues?
6. Does this checkin involve any code line that checks against numeric limits? Have you added test cases that actually test those limits?
7. Does this checkin allocate strings?
   a. Do you allocate memory for strings?
   b. If so, have you allocated memory to hold the string terminating null?
8. Does this checkin manipulate strings/arrays?
   a. Have you checked that all your accesses are within bounds?
   b. Is there any loop traversal over the string/array?
   c. If so, have you verified that there is no 'off by one' error for loop bounds?
9. Does this checkin involve any logical AND/OR/XOR operations using hardcoded masks?
   a. Have you verified that the mask is adequate for the bits you want to extract?